

Matkul : Pengembangan Aplikasi Mobile
Dosen : Muhammad Aminul Akbar, S.Kom., M.T.
Modul : Mobile_3_Event, State, Imperatif-Deklaratif, dan Implisit Intent
Kelas : SI – D

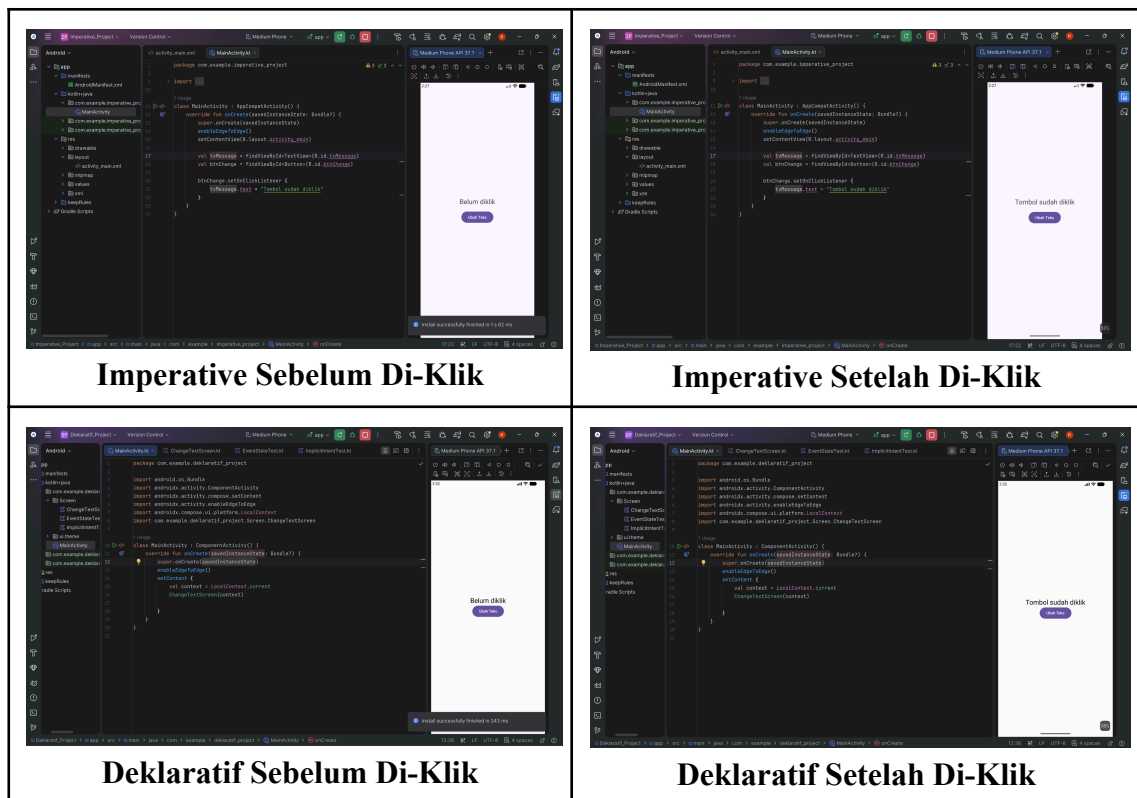
Nama Anggota : Raihan Nasrullah Alamin
NIM : 245150401111031

Percobaan

Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

Berikut untuk hasil screenshot dari Android Studio saya :



2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Pada project XML/View, objek yang diubah secara langsung ketika tombol diklik adalah objek TextView dengan nama **tvMessage**. Perubahan tersebut dilakukan melalui baris

```
tvMessage.text = "Tombol sudah diklik",
```

Cara ini disebut imperatif karena kita memberikan perintah secara langsung mengenai bagian UI mana yang harus diubah.

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Pada project Jetpack Compose, nilai yang diubah ketika tombol diklik adalah **state message**. Perubahan dilakukan melalui baris

```
message = "Tombol sudah diklik".
```

Alasannya, pada Compose tampilan dibuat berdasarkan state yang ada, sehingga ketika state berubah, tampilan yang menggunakan state tersebut akan ikut diperbarui.

4. Mengapa pada Compose tidak terdapat perintah seperti tvMessage.text atau setText()?

Pada Compose tidak ada perintah seperti tvMessage.text atau setText() karena Compose menggunakan konsep deklaratif. Artinya, kita tidak perlu memberi perintah satu per satu untuk mengubah tampilan. Kita cukup menentukan bahwa Text menampilkan message. Ketika message berubah, Compose akan mengetahui bahwa tampilan yang menggunakan nilai tersebut perlu diperbarui. Jadi, kita lebih fokus pada “tampilannya harus seperti apa”, bukan “bagaimana cara mengubah objek tampilannya”.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Pada XML/View, alurnya adalah event -> View -> perubahan tampilan. Ketika tombol ditekan, terjadi sebuah event yang ditangani oleh `setOnClickListener`. Setelah

itu, program langsung mengubah View, yaitu `tvMessage`, menggunakan `tvMessage.text`. Karena teks pada `TextView` sudah diubah, tulisan yang terlihat di layar juga langsung berubah menjadi “Tombol sudah di klik”. Jadi, perubahan UI dilakukan secara langsung oleh program.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Pada Compose, alurnya adalah event -> state -> recomposition -> perubahan tampilan. Ketika tombol ditekan, event tersebut mengubah state message. Setelah state berubah, Compose melakukan recomposition, yaitu proses Compose menyusun atau memperbarui kembali bagian UI yang dipengaruhi oleh state tersebut. Karena Text menggunakan message, maka teks yang ditampilkan otomatis berubah menjadi “Tombol sudah di klik”. Jadi, kita tidak perlu mengubah Text secara langsung.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum di klik"`? Jelaskan penyebabnya.

Jika pada Compose message hanya dibuat sebagai variabel biasa seperti **`var message = "Belum diklik"`**, perubahan nilainya tidak otomatis membuat tulisan di layar berubah. Hal ini karena Compose tidak mengetahui bahwa variabel tersebut berhubungan dengan tampilan yang perlu diperbarui. Berbeda dengan message yang dibuat sebagai state, perubahan pada state akan diketahui sehingga tampilan dapat menyesuaikan. Jadi, penggunaan state penting dalam pendekatan deklaratif karena perubahan nilai menjadi dasar untuk memperbarui tampilan.

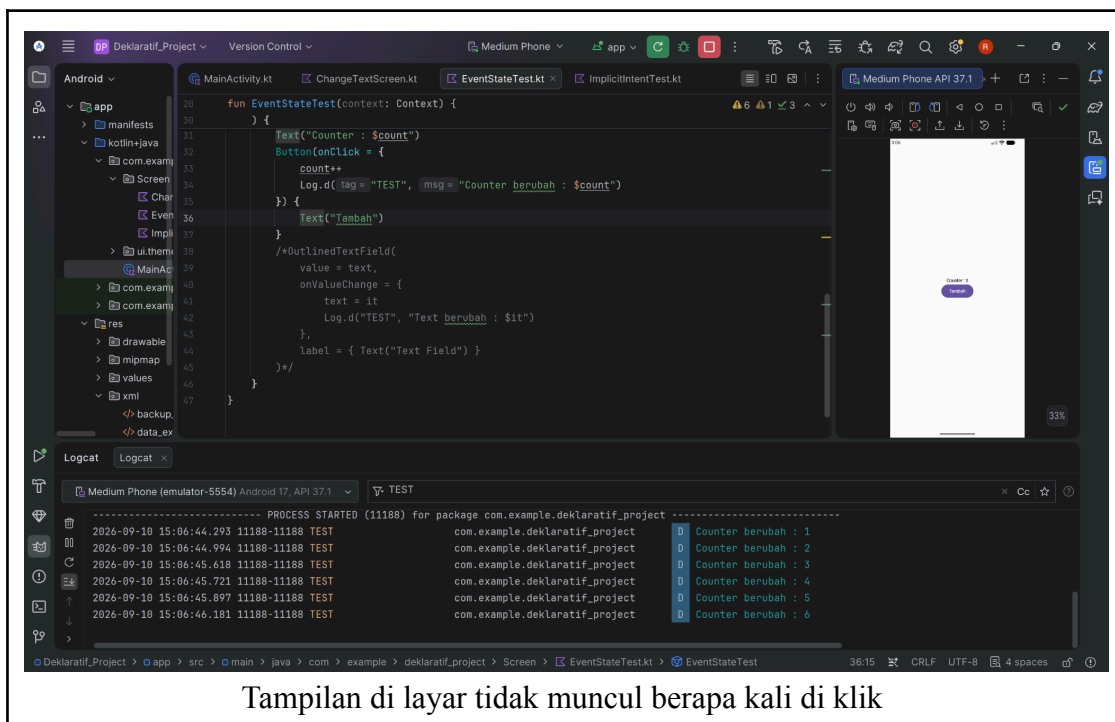
8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Dari percobaan ini, pendekatan imperatif dan deklaratif sama-sama bisa menghasilkan tampilan yang sama, tetapi cara kerjanya berbeda. Pada imperatif, seperti XML/View, kita secara langsung memberitahu program bagian tampilan mana yang harus diubah, contohnya `tvMessage.text` diubah menjadi teks baru. Sedangkan pada deklaratif, seperti Jetpack Compose, kita cukup mengubah nilai message yang menjadi dasar tampilan, lalu tampilan akan menyesuaikan dengan nilai tersebut. Jadi,

sederhananya, imperatif lebih berfokus pada “apa yang harus dilakukan terhadap tampilan”, sedangkan deklaratif lebih berfokus pada “tampilan seperti apa yang harus ditampilkan berdasarkan datanya”. Keduanya sama-sama baik, tinggal disesuaikan dengan teknologi dan kebutuhan aplikasi yang sedang dibuat.

Event Handling State

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" di klik?

Setiap kali tombol “Tambah” di klik, LogCat akan menampilkan pesan seperti Counter berubah : 1, kemudian Counter berubah : 2, Counter berubah : 3, dan

seterusnya. Angka tersebut bertambah setiap kali tombol ditekan karena pada kode terdapat perintah `count++`.

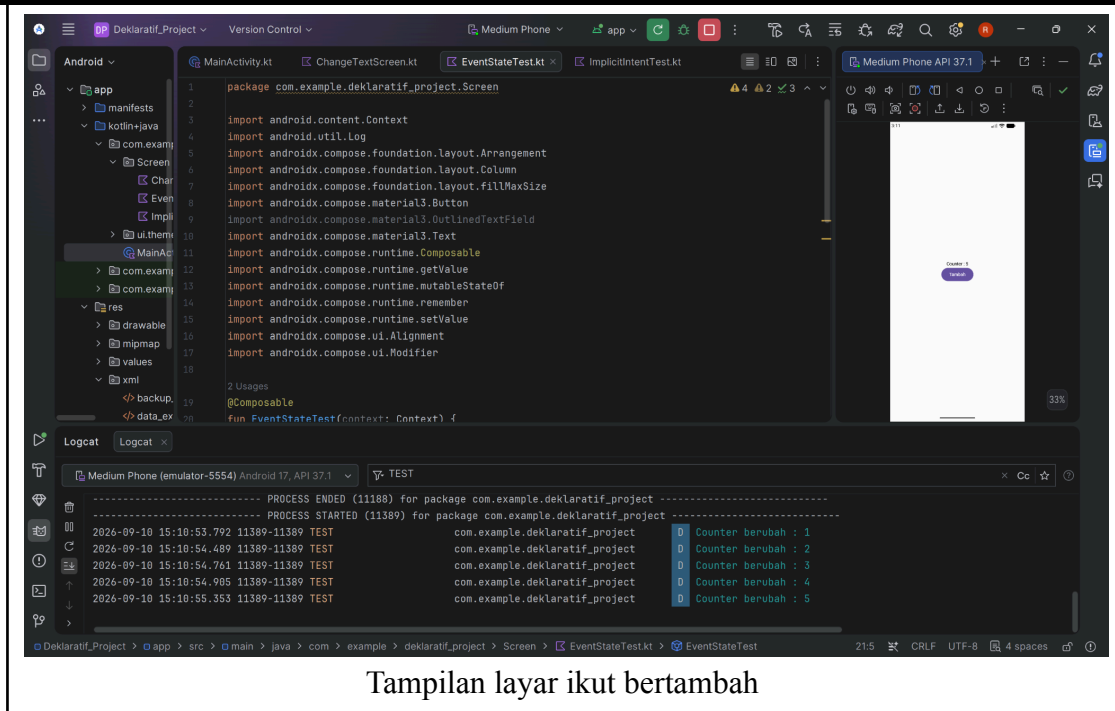
3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" di klik?

Pada percobaan pertama, tampilan aplikasi tidak mengalami perubahan angka dan biasanya tetap menampilkan "Counter : 0", walaupun tombol "Tambah" sudah ditekan beberapa kali. Jadi, angka yang muncul di layar tidak mengikuti angka yang terlihat di LogCat.

4. Apa yang menyebabkan hal tersebut terjadi?

Hal tersebut terjadi karena `count` masih dibuat sebagai variabel biasa dengan `var count = 0`. Variabel tersebut memang nilainya bertambah ketika tombol ditekan, tetapi Compose tidak mengetahui bahwa nilai tersebut berubah. Akibatnya, tampilan `Text("Counter : $count")` tidak diperbarui. Sementara itu, LogCat tetap bisa menampilkan angka yang bertambah karena perintah `Log.d()` dijalankan setiap kali tombol ditekan.

8. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" di klik beberapa kali.



Tampilan layar ikut bertambah

9. Apa yang muncul di LogCat setiap kali tombol "Tambah" di klik?

Pada percobaan kedua, LogCat akan menampilkan pesan yang angkanya bertambah setiap kali tombol ditekan, misalnya Counter berubah : 1, Counter berubah : 2, Counter berubah : 3, dan seterusnya. Hasil ini sebenarnya sama dengan percobaan pertama karena perintah untuk mencatat ke LogCat masih sama.

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" di klik?

Pada percobaan kedua, tampilan aplikasi akan ikut berubah setiap kali tombol ditekan. Jika awalnya "Counter : 0", setelah tombol ditekan menjadi "Counter : 1", kemudian "Counter : 2", "Counter : 3", dan seterusnya. Jadi, angka yang ada di tampilan sekarang mengikuti jumlah tombol yang sudah ditekan.

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

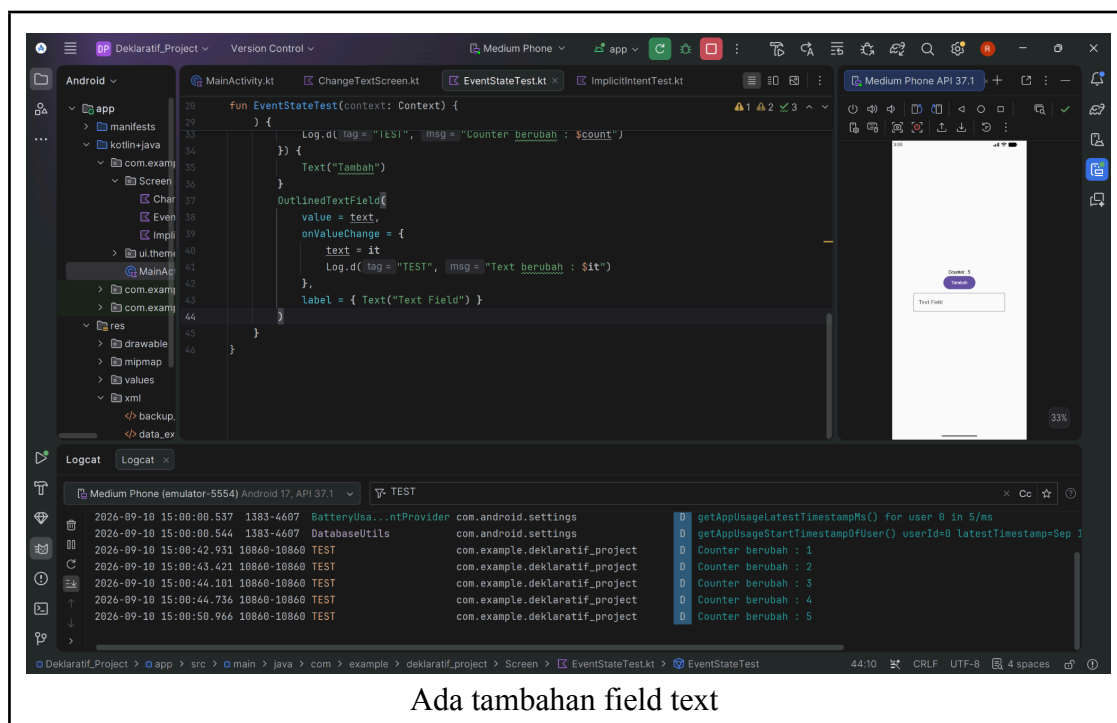
Perbedaannya ada pada cara count dibuat. Pada percobaan pertama, count hanya merupakan variabel biasa sehingga perubahan nilainya tidak membuat tampilan ikut diperbarui. Pada percobaan kedua, count dibuat menggunakan remember {

`mutableStateOf(0) }`, sehingga perubahan nilai `count` dapat diketahui oleh Compose. Ketika nilainya berubah, Compose memperbarui tampilan yang menggunakan `count`. Inilah alasan mengapa pada percobaan kedua angka di layar dapat berubah.

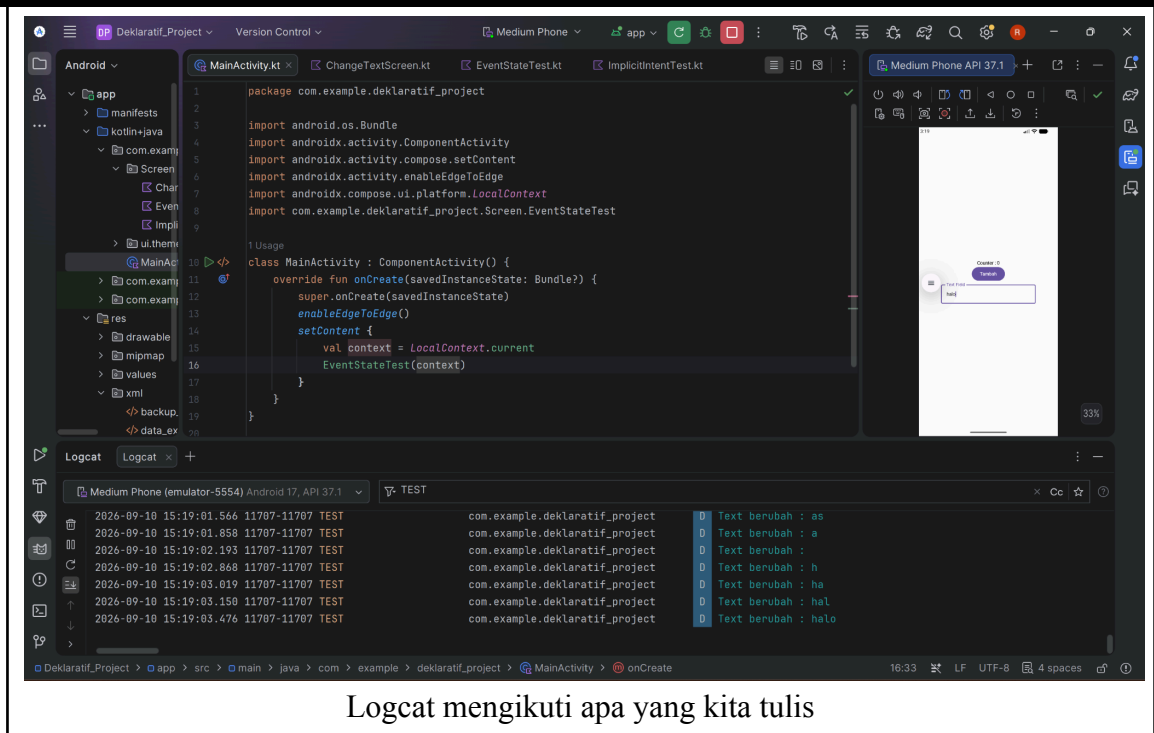
12. Apa yang dimaksud recomposition pada Compose?

Recomposition adalah proses ketika Compose memperbarui kembali bagian tampilan yang menggunakan data yang berubah. Misalnya, ketika nilai `count` berubah dari 2 menjadi 3, Compose mengetahui bahwa `Text` yang menampilkan `count` perlu diperbarui. Akhirnya, tulisan di layar berubah dari “Counter : 2” menjadi “Counter : 3”. Jadi, sederhananya, recomposition adalah proses Compose menyesuaikan tampilan ketika state yang digunakan oleh tampilan tersebut berubah.

14. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" di klik beberapa kali.



15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?



16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

17. Apa yang menyebabkan hal tersebut terjadi?

Hal tersebut terjadi karena text dibuat sebagai state menggunakan remember { mutableStateOf("") }. Ketika isi Textfield berubah, bagian onChangeValueState dijalankan dan nilai text diperbarui dengan text = it. Karena text merupakan state, Compose mengetahui bahwa nilainya berubah dan kemudian menyesuaikan tampilan Text Field dengan nilai terbaru.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Pada program terakhir terdapat dua state, yaitu count dan text. count digunakan untuk menyimpan jumlah berapa kali tombol "Tambah" ditekan, sedangkan text digunakan untuk menyimpan tulisan yang dimasukkan ke dalam Text Field.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

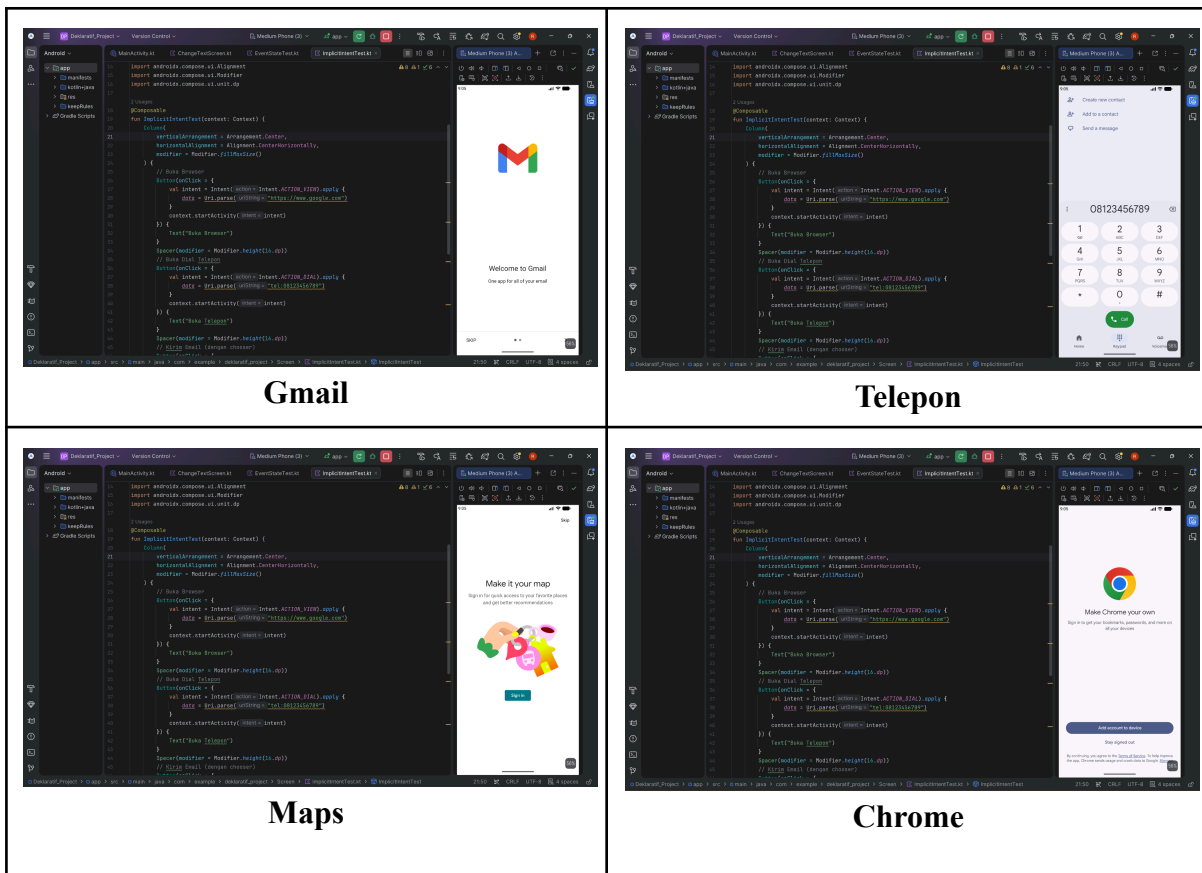
Terdapat dua event utama, yaitu event ketika tombol “Tambah” ditekan dan event ketika isi Text Field berubah. Event tombol terdapat pada Button(onClick = { ... }), sedangkan event perubahan Text Field terdapat pada onValueChange = { ... }.

20. Kapan event - event tersebut dijalankan?

Event pada tombol “Tambah” dijalankan setiap kali pengguna menekan tombol tersebut. Ketika event ini terjadi, nilai count bertambah satu dan pesan perubahan ditampilkan di LogCat. Sementara itu, event pada Text Field dijalankan setiap kali pengguna memasukkan, menghapus, atau mengubah huruf di dalam Text Field. Ketika event tersebut terjadi, nilai text akan disesuaikan dengan isi terbaru dari Text Field dan perubahan tersebut juga dicatat di LogCat.

Implisit Intent

6. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya



11. Apa perbedaan fungsi dari masing-masing action intent tersebut?

Intent.ACTION_VIEW digunakan untuk menampilkan atau membuka suatu data menggunakan aplikasi yang sesuai. Pada kode, action ini digunakan untuk membuka browser melalui URL dan membuka Maps melalui koordinat lokasi.

Intent.ACTION_DIAL digunakan untuk membuka aplikasi telepon pada tampilan dialer dengan nomor yang sudah diberikan. Intent ini tidak langsung melakukan panggilan.

Intent.ACTION_SENDTO digunakan untuk mengirim data ke tujuan tertentu, dalam kasus ini alamat email menggunakan mailto:. Sistem kemudian dapat menampilkan aplikasi email yang sesuai.

16. Apa fungsi data pada intent ?

data berfungsi untuk menentukan data atau tujuan yang akan diproses oleh Intent. Data biasanya dinyatakan dalam bentuk Uri, misalnya URL untuk browser, nomor telepon untuk dialer, alamat email, atau koordinat lokasi untuk Maps.

Contohnya:

`https://www.google.com` → membuka halaman web.

`mailto:test@gmail.com` → menentukan alamat email tujuan.

`geo:-7.250445,112.768845` → menentukan koordinat lokasi.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Action dan Data bekerja bersama untuk menentukan aplikasi yang dapat menangani sebuah Intent. Action menunjukkan apa yang ingin dilakukan, sedangkan Data menunjukkan terhadap data apa tindakan tersebut dilakukan.

Contohnya, **ACTION_VIEW** + URL akan membuat Android mencari aplikasi yang dapat menampilkan URL, seperti browser. Sementara **ACTION_VIEW** + `geo:` membuat Android mencari aplikasi yang dapat menampilkan lokasi, seperti Google Maps. Jadi, kombinasi Action dan Data menentukan aplikasi yang dianggap cocok oleh sistem Android untuk menjalankan Intent tersebut.

